Comprehensive Application

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

This tutorial uses the factory road sign model and lane line recognition model to perform centered movement on the sandbox map, while recognizing road signs and performing corresponding sign function actions.

2. Program Path

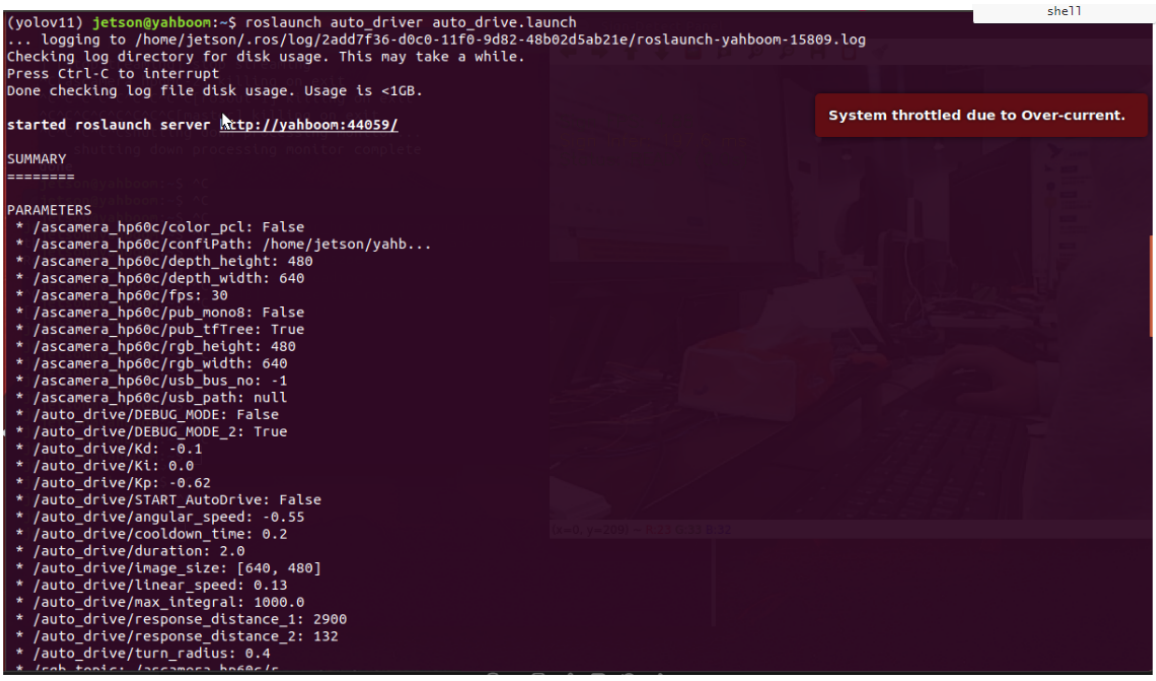
Autonomous driving source code path:
Lane line and road sign recognition path:
`/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py`

3. Running Example Program

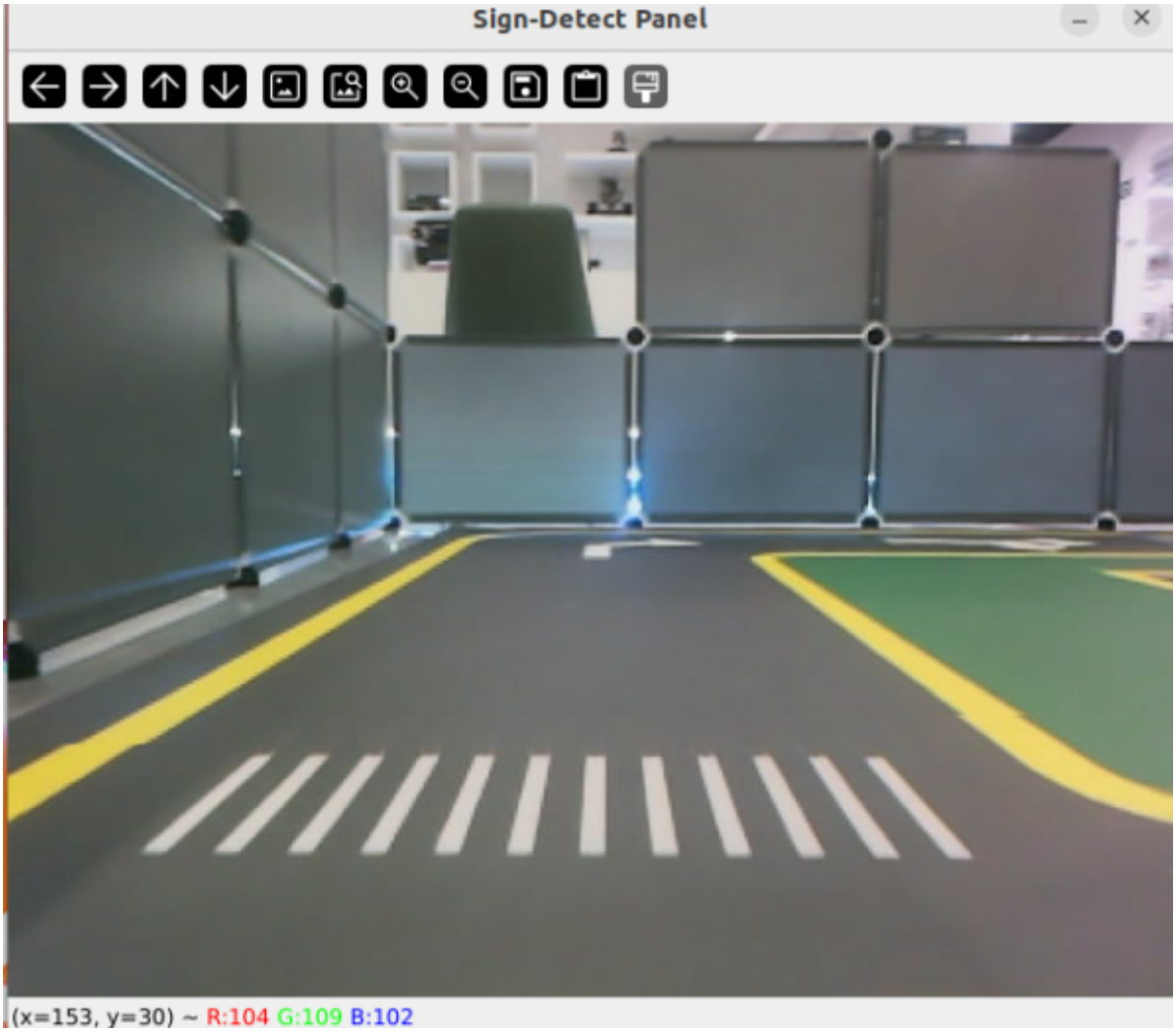
3.1 Running Program

- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

```
roslaunch auto_driver auto_drive.launch
```



After successful startup, the display will show but the car will not move, for testing road sign recognition. (If you want the car to move, change the parameter START_AutoDrive in auto_drive_node.yaml to True)



- Parameter configuration

Parameter path:
`/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml`

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: False
DEBUG_MODE_2: True
START_AutoDrive: True
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

After modifying parameters, restart the launch file to use the modified parameters.

Parameter analysis:

【turn_radius】: Turning radius. Normally no need to change this parameter

【drive_speed】: Linear speed magnitude. The car's driving speed, factory default is 0.13. If you modify this parameter, you need to debug other parameters accordingly

【angular_speed】: Turning angular speed magnitude

【duration】: Right turn time when recognizing right turn sign

【cooldown_time】: Road sign recognition cooldown time

【DEBUG_MODE】: Lane line visualization debug parameter, turning it on may affect centering effect

【DEBUG_MODE_2】: Road sign visualization debug parameter, turning it on may affect centering effect

【START_AutoDrive】: Control car movement parameter, true means car can move

【kp、ki、kd】: Car centering PID parameters

3.2 Car movement after recognizing road signs

【straight】: Car drives straight according to default linear speed.

【turnright】: Car will stop for one second, then turn according to fixed actions. Note that the right turn sign should be placed in the blue box near the parking lot

【honking】: Horn sounds for one second

【speedlimit】: Decelerate for three seconds then resume normal driving

【stop】: Stop for three seconds then resume normal driving

【school】: Decelerate for three seconds then horn sounds for one second

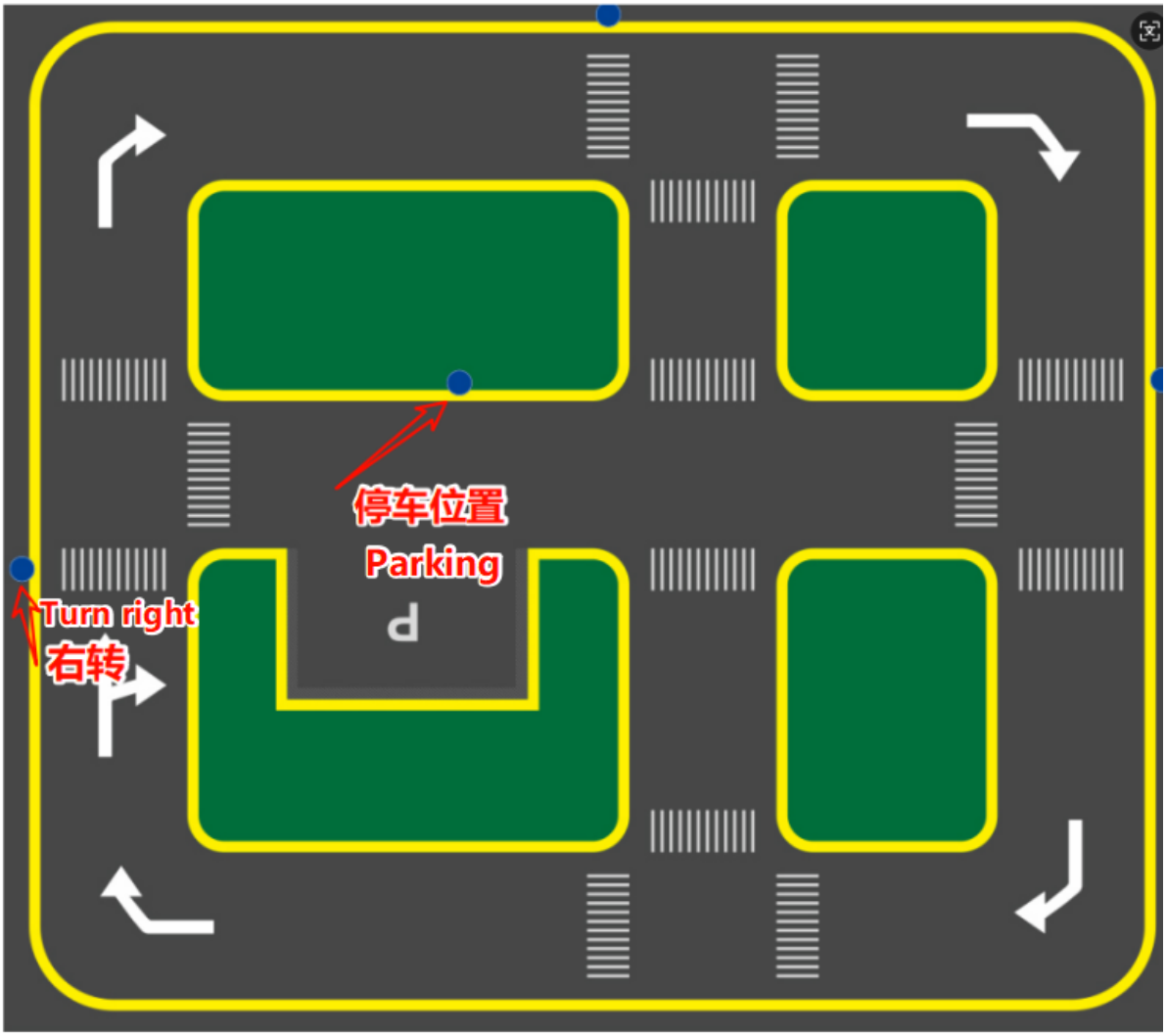
【parkA、parkB】: Parking fixed action, after which the car stops, needs 【straight】 【speedlimit】 to unlock to resume normal driving

【redlight】: Stop and do not move

【greenlight】: Car continues driving

【yellowlight】: Stop for three seconds then resume normal driving

The following image shows the fixed positions for right turn and parking. Because the program uses fixed actions, for better results it is recommended to place them in these positions. If you need to place them in other positions, you need to re-debug the fixed action timing.



3.3 Debugging Parameters

Here are methods and experiences for debugging parameters:

- Permanent parameter modification debugging

Modify parameter files to achieve permanent parameter changes. Parameter paths:

`/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml`

`/home/jetson/yahboomcar_auto/src/auto_driver/config/yolo_param.yaml`

`/home/jetson/yahboomcar_auto/src/auto_driver/config/midline_points.yaml`

The three parameters represent: autonomous driving parameters, road sign recognition parameters, lane line centering parameters

Autonomous driving parameters:

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: False
DEBUG_MODE_2: True
START_AutoDrive: True
```

```
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

The above parameters have the same meaning as dynamically adjusting parameters. After modification here, the next run will call the program here by default. Note that START_AutoDrive: True determines whether the car can run.

Road sign recognition parameters:

```
#Road sign detection inference parameters
sign_detection:
  model_path: '/home/jetson/yahboomcar_auto/src/sign_model.engine'
  conf: 0.92   #Set the minimum confidence threshold for detection. Detected
objects with confidence below this threshold will be ignored. Adjusting this
value helps reduce false positives.
  iou: 0.7     #Overlap threshold for non-maximum suppression (NMS). Lower values
reduce detection results by eliminating overlapping boxes
  rect: True    #If enabled, minimally pad the shorter side of the image until
divisible by stride to improve inference speed.
  half: True     #If enabled, use half-precision inference, which can speed up
model inference on supported GPUs with minimal impact on accuracy.
  device: "cuda:0"      #Inference device
  batch : 50         #Batch size, larger batch sizes can provide higher
throughput, thus reducing total inference time.
  max_det : 2         #Maximum number of objects to detect
  augment : False     #Enable test-time augmentation (TTA) for prediction, which
may improve detection robustness but reduces inference speed.
  verbose : False      #Whether to output detailed inference information in
terminal.
  classes : [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]

line_detection:
#Lane line detection inference parameters
  model_path: '/home/jetson/yahboomcar_auto/src/lane_V6.engine'
  conf: 0.5      #Set the minimum confidence threshold for detection. Detected
objects with confidence below this threshold will be ignored. Adjusting this
value helps reduce false positives.
  iou: 0.6       #Overlap threshold for non-maximum suppression (NMS). Lower
values reduce detection results by eliminating overlapping boxes
  rect: True     #If enabled, minimally pad the shorter side of the image until
divisible by stride to improve inference speed.
  half: True      #If enabled, use half-precision inference, which can speed up
model inference on supported GPUs with minimal impact on accuracy.
  device: "cuda:0"     #Inference device
  batch : 50       #Batch size, larger batch sizes can provide higher throughput,
thus reducing total inference time.
  max_det : 4       #Maximum number of objects to detect
  augment : False     #Enable test-time augmentation (TTA) for prediction, which
may improve detection robustness but reduces inference speed.
  verbose : False     #Whether to output detailed inference information in
terminal.
```

The above parameters can use factory defaults, generally no need to modify

Lane line centering parameters

```
midline_points:
  start_point:
    x: 360
    y: 477
  end_point:
    x: 355
    y: 424
  midline_length: 120
  mid_distance: [200,40]
```

start_point and end_point calibrate the car's center point, specific details can be found in the center axis calibration course. The midline_length parameter is used to adjust the car's turning timing on the sandbox map, smaller values turn slower, conversely larger values turn earlier

Debugging experience:

1. If lane centering is unstable, refer to the "4. Lane keeping in autonomous driving" chapter
2. If right turn recognition is insufficient or excessive, adjust the duration parameter to control right turn time
3. To prevent frequent road sign recognition, modify cooldown_time road sign cooldown, indicating the cooling duration between recognizing road signs
4. If lane centering adjustment is too slow, increase kp.

4. Source Code Analysis

4.1、auto_drive.py

Program source code path
`/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py`

Lane line control process is divided into

1. Receive and process image data
2. Detect lane lines and traffic signs
3. Decide control strategy based on detection results
4. Publish control commands (speed, direction)
5. Loop through the above steps

```
def _handle_lane_detect_result(self, left_box, right_box, frame):
        '''Handle lane detection results and control vehicle'''
        error_angular = [0.0, 0.0]
        if self.flag != False:
            return frame
        # Classify processing based on detected lane line conditions
        if left_box is not None and right_box is not None and left_box[0] <
right_box[0]:
            # Both sides have lane lines
            self.error_mode = 0
            frame = self.lane_detect.draw_left_line(frame, left_box)
            frame = self.lane_detect.draw_right_line(frame, right_box)
            frame, x_center, y_center = self.lane_detect.draw_center_point(frame,
left_box, right_box)
            error_angular = self.lane_detect.cal_error_angular(x_center,
y_center)
```

```
            # PID control
            if abs(error_angular[1]) >= 0.03 and abs(error_angular[1]) < 0.3:
                angular_z = self.pid_controller.pid_control(error_angular[1])
                angular_z = max(min(angular_z, 0.3), -0.3)
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=angular_z)
            else:
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=0.0)

        elif left_box is not None and right_box is None and left_box[0] < 320:
            # Only left lane line
            self.error_mode = 1
            self.lane_detect.prev_center = None
            frame = self.lane_detect.draw_left_line(frame, left_box)

            if_infer = self.lane_detect.IF_lane_intersection(left_box)
            if if_infer is not None:
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=self.angular_speed)
            else:
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=0.0)

        elif right_box is not None and left_box is None:
            # Only right lane line
            self.error_mode = 2
            self.lane_detect.prev_center = None
            frame = self.lane_detect.draw_right_line(frame, right_box)
            self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
        else:
            # No lane lines detected
            self.error_mode = 99
            self.lane_detect.prev_center = None
            self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)

        # Display angle deviation
        cv2.putText(frame, "Angle Dev: {:.2f} rad".format(error_angular[1]), (10,
105),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 255), 2)
        frame = self.lane_detect.draw_midline(frame)

        return frame
```

Road sign recognition processing control process is divided into

1. **Image callback** (image_callback):
    - Convert ROS image messages to OpenCV format
    - Put images into processing queue
2. **Sign detection processing** (handle_sign):
    - Get image frames from queue
    - Use YOLO model for object detection

- Process detection results:
  - Filter ignored signs (such as "turnright_ground", "sidewalk_ground")
  - Publish detected sign information
  - Display FPS and inference time in debug mode
  - Display detection results in real-time

```
def _sign_detection_worker(self):
        '''Sign detection worker thread'''
        sign_pub_counter = 0

        while not rospy.is_shutdown() and not self._shutdown_flag:
            if self.cv_image is None:
                time.sleep(0.01)
                continue

            # Check cooldown status
            current_time = time.time()
            if self.sign_cooldown:
                if current_time - self.last_sign_time >= self.cooldown_duration:
                    self.sign_cooldown = False
                    rospy.loginfo("Sign detection cooldown ended")
                else:
                    # During cooldown period, skip sign detection
                    time.sleep(0.01)
                    continue

            try:
                frame = self.cv_image.copy()
                infer_start = time.time()

                # Perform mark detection
                results = self.sign_detect.get_infer_result(frame, True)
                annotated_frame = frame.copy()

                # Record whether a sign that requires cooldown was detected
                detected_sign_requires_cooldown = False

                for res in results:
                    if res.boxes is not None:
                        annotated_frame = res.plot()
                        boxes = res.boxes
                        for i in range(len(boxes)):
                            box_coords = boxes.xyxy[i].tolist()
                            class_name = res.names[int(boxes.cls[i].item())]
                            confidence = boxes.conf[i].item()

                            # Check if it's an ignored sign
                            if class_name in self.ignored_signs:
                                # rospy.loginfo(f"Ignoring sign: {class_name}")
                                continue  # Skip ignored signs

                            # 发布检测结果
                            if sign_pub_counter % 5 == 0:
                                detection_msg = DetectionObject()
                                detection_msg.class_name = class_name
                                detection_msg.confidence = confidence
```

```
                                detection_msg.xmin = float(box_coords[0])
                                detection_msg.ymin = float(box_coords[1])
                                detection_msg.xmax = float(box_coords[2])
                                detection_msg.ymax = float(box_coords[3])
                                self.sign_publisher.publish(detection_msg)

                                # Add detection result to event queue (ignored
signs not added)
                                if not self.action_running:
                                    object_distance =
self._get_distance(float(box_coords[1]))
                                    obj = Object(class_name, confidence,
object_distance)
                                    try:
                                        self.event_queue.put_nowait(obj)
                                    except queue.Full:
                                        pass

                                    # Set cooldown flag (except for ignored
signs, other signs all trigger cooldown)
                                    detected_sign_requires_cooldown = True

                            sign_pub_counter += 1

                # If a sign that requires cooldown is detected, start cooldown
                if detected_sign_requires_cooldown:
                    self.sign_cooldown = True
                    self.last_sign_time = current_time
                    rospy.loginfo(f"Sign detected, starting
{self.cooldown_duration}s cooldown")

            except Exception as e:
                rospy.logerr("Error in sign detection: %s", str(e))
                time.sleep(0.01)
```

The main thread is responsible for receiving images, while worker threads handle image processing and detection tasks, communicating between threads through queues.

4.2、yolov11_infer.py

Program source code path
`/home/jetson/yahboomcar_auto/src/auto_driver/scripts/utils/yolov11_infer.py`

Control process

Sign_Detect class:

- When creating a Sign_Detect instance, load and parse configuration file
- Call init_yolo_engine() method:
  - Initialize image queue (maximum capacity 10)
  - Set model inference parameters (confidence, IOU thresholds, etc.)
  - Load YOLO model
- Put images into processing queue through put_image method
- Get inference results through get_infer_result method
- Use YOLO model for object detection

Lane_Detect class:

- **Lane line detection**: Use YOLO model to detect lane lines in images
- **Lane center point calculation**: Calculate center point of left and right lane lines
- **Lane departure detection**: Calculate vehicle's offset angle and distance relative to lane
- **Visualization**: Provide multiple drawing methods for displaying detection results

```
class Sign_Detect(YOLO_MODEL):
    """Road sign detection"""
    def __init__(self,config_file_path):
        with open(config_file_path, "r") as file:
            full_config = yaml.safe_load(file)
            self.config_param = full_config.get("sign_detection", {})
...

class Lane_Detect(YOLO_MODEL):
    """Lane line detection model"""
    def __init__(self,config_file_path:str,
                 midline_file_path:str):
        with open(config_file_path, "r") as file:
            full_config = yaml.safe_load(file)
            self.config_param = full_config.get("line_detection", {})
        with open(midline_file_path, "r") as file:
            self.point_config = yaml.safe_load(file)
...
```